

ALUminati

High Speed 4-Bit ALU
ECE 4740 Final Project

Chimdi Anude (cca54), Zarif Karim (zk67), Amber Li (adl94),
Anushka Mathew (asm367), Vidhula Pallavor (vp278),
Nicholas Papapanou (ngp37), Kaniskaa Mohan Sangeetha (km2224)

ABSTRACT

For our final project, we designed a fast 4-bit ALU that is able to perform 10 functions. It consists of several subcircuits to implement each individual function, as well as control logic to tie it all together at the top level. Our primary focus was to utilize dynamic logic to increase the speed of the ALU, with a secondary focus on implementing alternative logic to save on power.

INTRODUCTION AND MOTIVATION

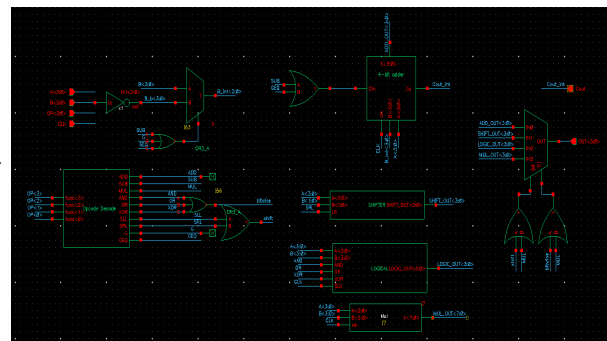
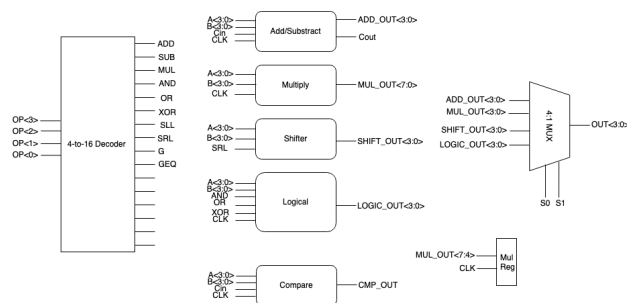
We were two separate groups that expressed interest in designing an ALU. The motivation for this project was to connect what we learned this semester in VLSI to a familiar digital circuit. However, the challenge for us was to make it faster. This allowed us to explore alternative ways to do combinational logic, including pass gates and dynamic logic. Our ALU performs the following types of operations: arithmetic (ADD, SUB, MUL), bitwise (AND, OR, XOR), logical shifts (SLL, SRL), and comparison (G, GEQ).

THEORY OF OPERATION

Our ALU allows one to enter two 4-bit inputs and a 4-bit opcode to select a function to perform on those inputs. It will return the result of this calculation as either a 4-bit output or a single bit. We use dynamic logic to implement a fast ALU, optimizing primarily for delay and secondarily for power consumption. This module integrates arithmetic, logical, shifting, and multiplication operations into a unified ALU architecture. A 4:16 decoder translates the opcode into control signals, directing data through the appropriate functional unit—adder, shifter, logic, or multiplier. Additional combinational logic handles operation selection and output multiplexing. A comparator is implemented using custom logic by manipulating the 4-bit adder's carry-in and carry-out signals.

TOP-LEVEL DESCRIPTION

On the top level, 4 blocks operate on the 2 4-bit inputs. They are selected based on the opcode, which uses a decoder to figure out what operation is to be performed. From there, the inputs either go to the 4-bit adder, shifter, logical or multiplier. Comparison operations are calculated through the adder. The ALU can only do one operation at a time. Once done, the output is an input to a mux and connected to the ALU's output.



Figures 1 and 2: Top-Level Block Diagram and Schematic

KEY SUBCIRCUITS AND THEIR LAYOUTS

Arithmetic Operations (ADD, SUB, MUL)

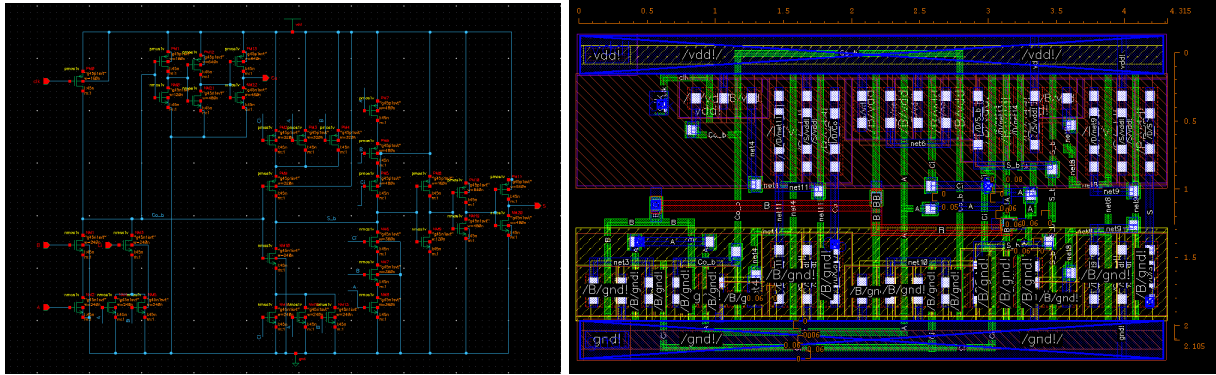
ADD

We designed a full adder where the carry path uses dynamic logic to achieve high speed, while the sum path uses static logic to avoid glitches. This design choice was made because dynamic gates are particularly sensitive to glitches for example, when inputs change from $A = 1, B = 0$, and $C_i = 0 \rightarrow 1$. By using dynamic logic for the carry and static logic for the sum, we observed a reduction in delay compared to a fully static implementation.

	$T_{PD-Rise} (ns)$	$T_{PD-Fall} (ns)$	Power (fJ/cycle)
Dynamic	0.097752	0.092105	53.5
Static	0.10235	0.102261	31.64

Table 1: Comparison of Dynamic and Static Full Adder

Although the improvements in delay appear small in isolation, we anticipated that the benefits would become more pronounced when scaling to a 4-bit adder. To implement the 4-bit adder, we used four of these full adders arranged in a Ripple Carry Adder configuration. Since the inputs are only 4 bits wide, more complex architectures like Carry Look-Ahead would not offer significant speed advantages, so we opted for the simpler Ripple Carry design. The initial carry-in (C_i) was set to 0.



Figures 3 and 4: Full Adder Schematic and Layout

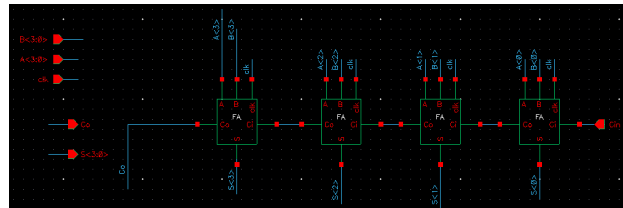


Figure 5: Ripple Carry Adder (ADD) Schematic

To ensure minimal glitches, we carefully sized the transistors for the dynamic circuit. Transistor sizing was optimized to balance drive strength and minimize delay while avoiding unnecessary power consumption. In particular, we buffered the outputs using $4\times$ inverters to sharpen the

transitions and isolate the logic from capacitive loads. This approach helped ensure clean and glitch-free outputs, which we confirmed through post-layout simulations even after parasitic extraction. The absence of glitches in the output validated our design strategy under realistic conditions.

We also experimented with inserting two additional buffers before the sum and carry outputs of the ripple carry adder, to improve performance (apart from the buffers already inside the sub-cell), but the difference in performance was not significant enough to downplay the increase in area due to 10 new inverters, so we discarded them.

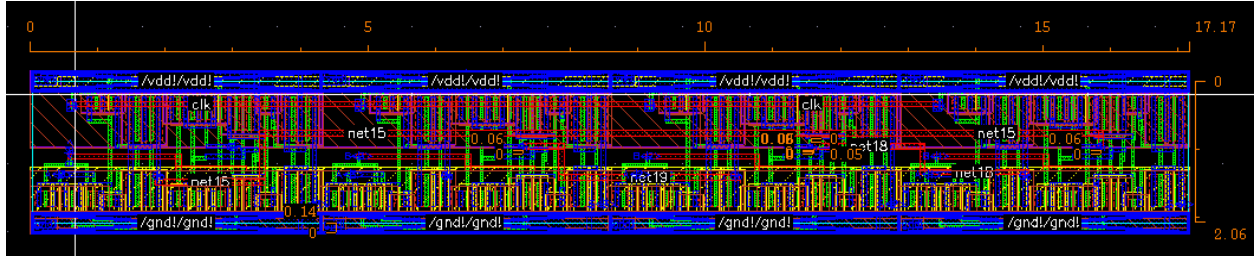
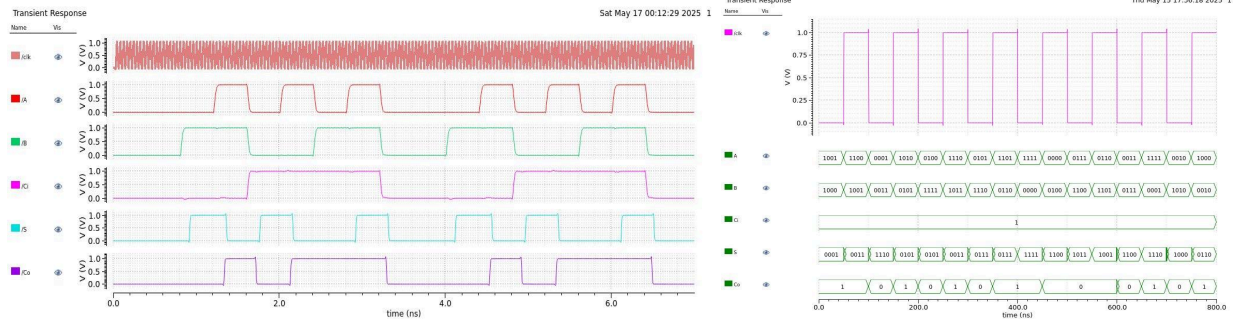


Figure 6: Ripple Carry Adder (ADD) Layout



Figures 7 and 8: Full Adder and Ripple Carry Adder (ADD) Verification

SUB

The ADD/SUB module shared the same datapath, with a control signal used to enable two modifications:

1. The second operand (B) is inverted.
2. The initial carry-in (Ci) is set to 1.

This configuration computes subtraction using two's complement arithmetic. For example, 1 - 3 (see the third input in Figure 9) would yield -2 in two's complement form. Additional logic may be needed to interpret or display the result in a human-readable signed form, by noticing when $Co = 0$.

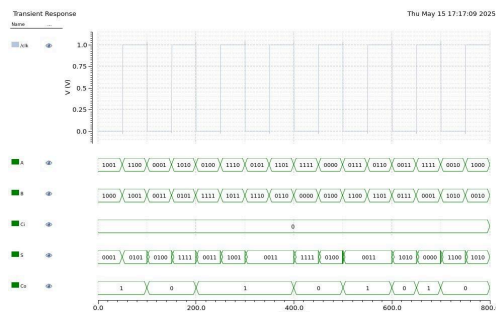
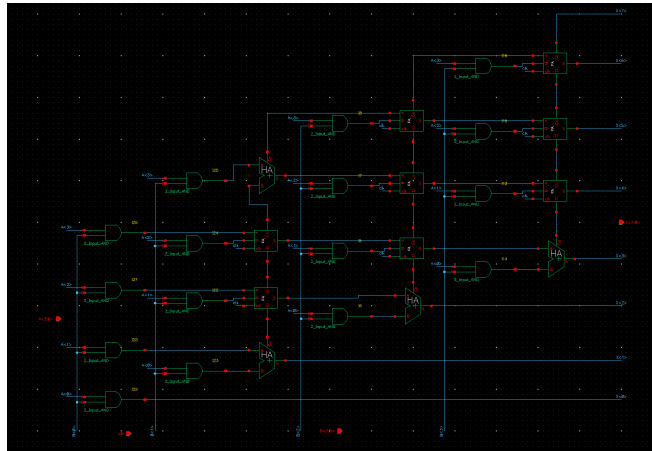


Figure 9: SUB Verification

MUL

This subcell implements a 4-bit array multiplier consisting of 8 dynamic full adders, 4 static half adders, and 16 AND gates. MUL is the only operation that produces an 8-bit output, so in the top level, the least significant four bits are wired to the ALU output mux, and the most significant four bits are sent to a designated register that the user can read to recover the entire output. As with ADD and SUB, verification was done via a custom SPICE stimulus file using PWLs to test all input combinations.



Figures 10 and 11: MUL Schematic and Layout

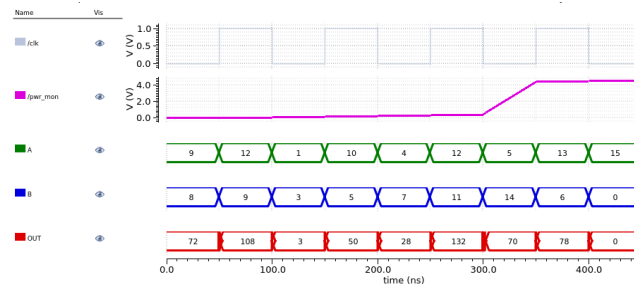
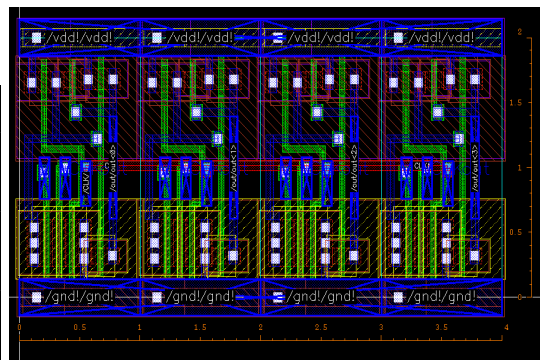
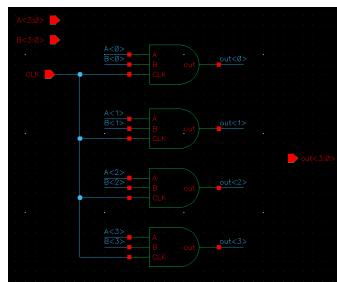
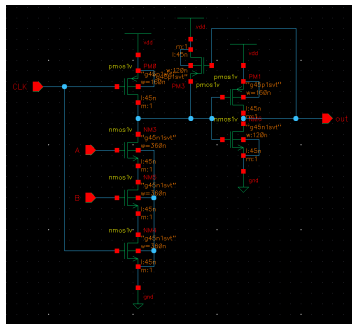


Figure 12: MUL Verification

Bitwise Operations (AND, OR, XOR)

AND

This subcell ANDs each bit of the 4-bit inputs together.



Figures 13, 14, and 15: AND schematic and layout

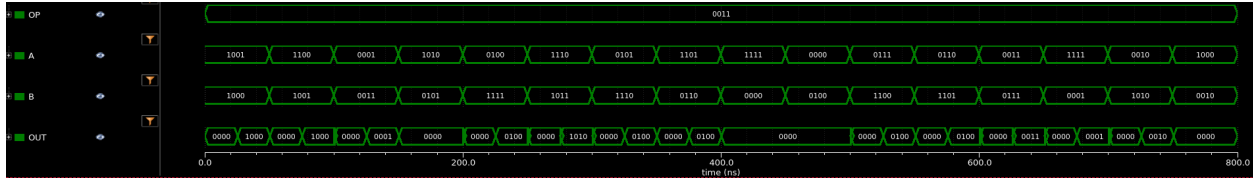
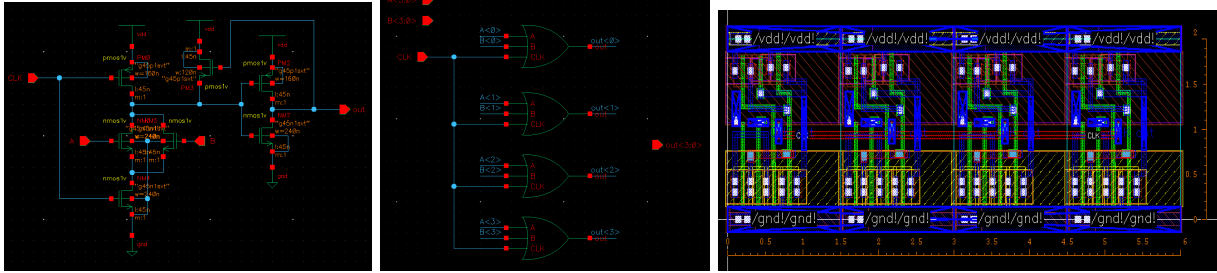


Figure 16: AND simulation

OR

This subcell ORs each bit of the 4-bit inputs together.



Figures 17, 18, and 19: OR schematic and layout

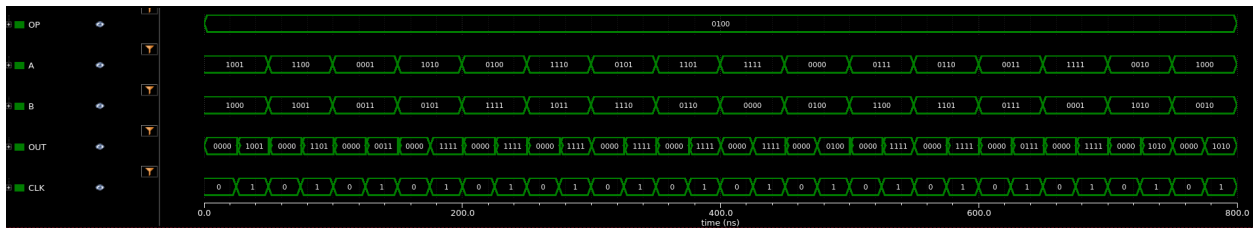
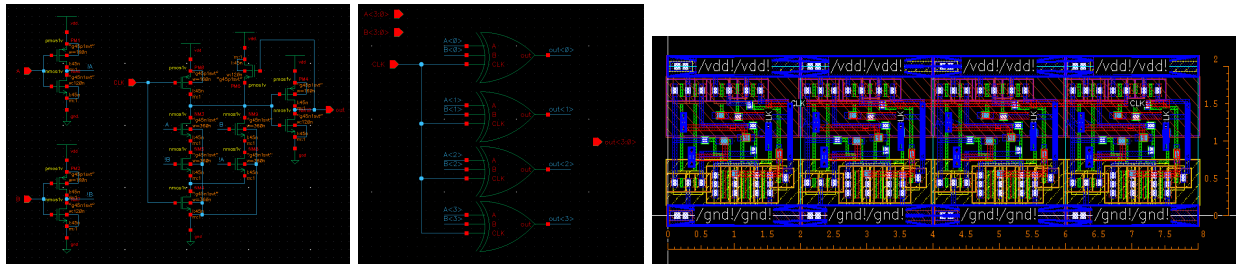


Figure 20: OR simulation

XOR

This subcell XORs each bit of the 4-bit inputs.



Figures 21, 22, and 23: XOR schematic and layout

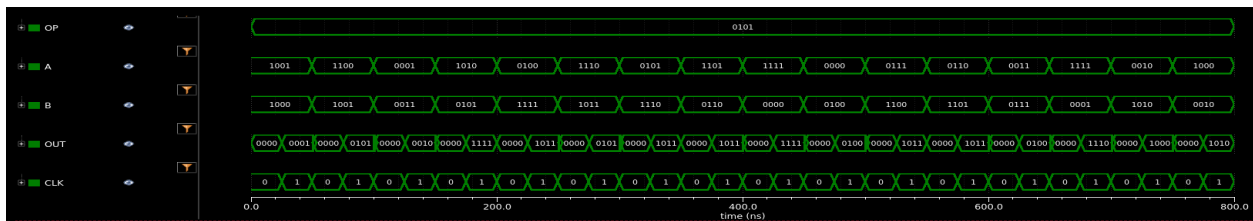


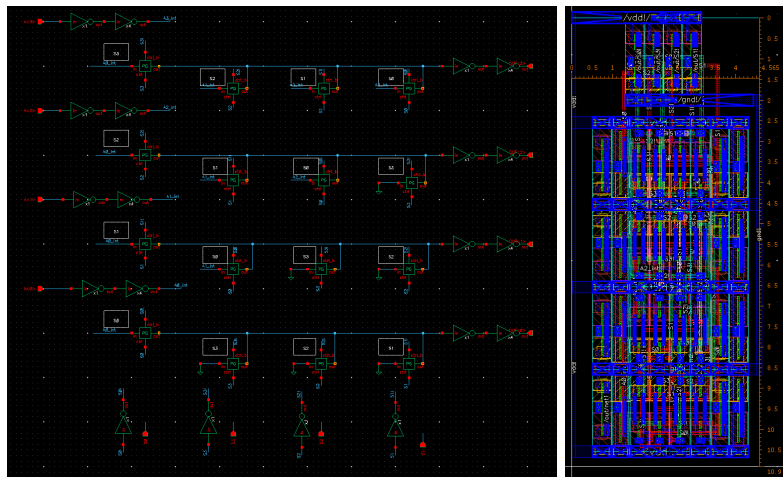
Figure 24: XOR simulation

These operations were optimized using dynamic logic by incorporating the CLK input. Since only nmos is used for the logic, a level restorer was included to help with pull-up. Like the

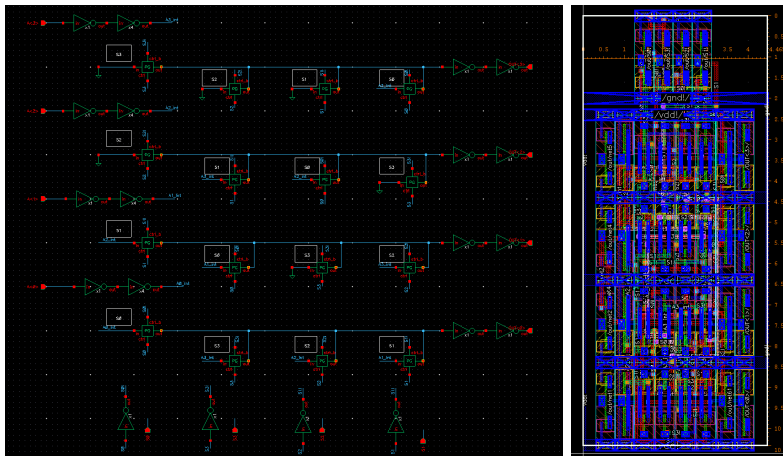
arithmetic operations, these three bitwise operations were verified via a custom SPICE stimulus file using PWLs to test all input combinations. (*Note: See appendix for bitwise top-level schematic and layout.)

Logical Shift Operations (SLL, SRL)

To improve upon shifting, we decided to design logical left and logical right barrel shifters. These shifters utilize a matrix of passgates and on-hot enables to shift the input either to the right or left by a specific number of bits. Though a decoder is required to select only one of the select lines, barrel shifters allow for a constant delay for all shift amounts. Barrel shifters take up less space than a mux shifter in layout as well, being both a speed and area win!



Figures 25 and 26: SLL Schematic and Layout



Figures 27 and 28: SRL Schematic and Layout

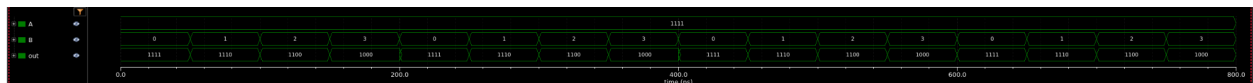


Figure 29: SLL Verification

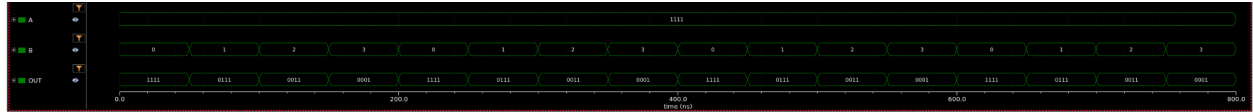


Figure 30: SRL Verification

Comparison Operations (G, GEQ)

The comparator utilizes subtraction logic to generate a single binary output. If $C_{in} = 0$, the G (greater than) operation is executed, returning a 1 if $A > B$ or a 0 otherwise. If $C_{in} = 1$, the GEQ (greater than or equal to) operation is executed, returning a 1 if $A \geq B$ or a 0 otherwise.

As the Comparator depends only on the carry out of our FA, which was optimized with dynamic logic, we see a slight speed up in execution of the G and GEQ functions. We incorporated the comparator into our adder block in the top level, but shown below is a schematic of it as its own subcell.

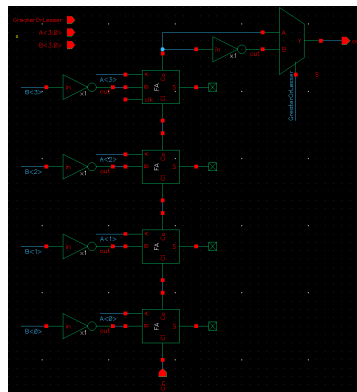


Figure 31: Comparator Schematic

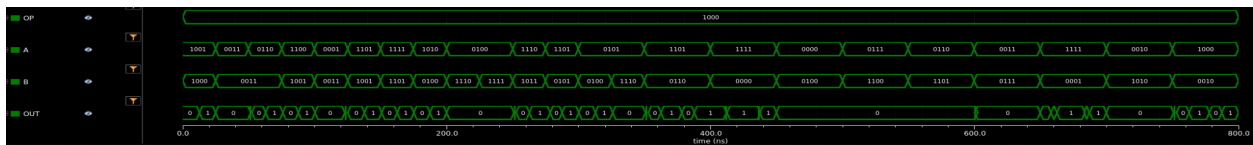


Figure 32: G Verification

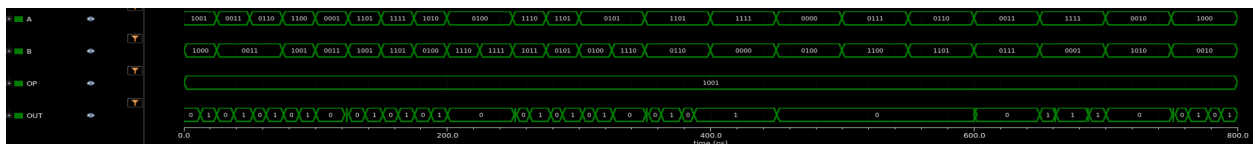
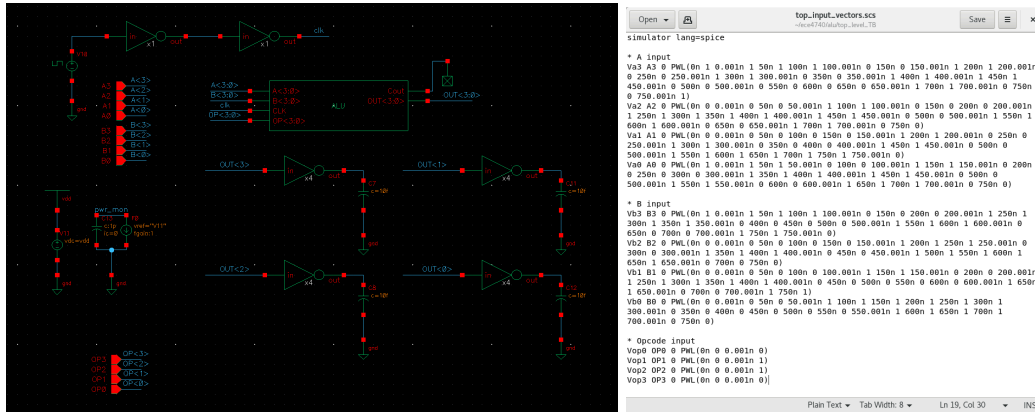


Figure 33: GEQ Verification

SIMULATION RESULTS

Subcells were verified individually in test benches, using custom SPICE stimulus files that initialized different combinations of input vectors using PWL (piecewise linear) voltage sources. Once each subcell's functionality was confirmed, it was incorporated into the top level, where we

ran similar tests to verify that the control logic was producing the correct output given a combination of input vectors and opcode.



Figures 34 and 35: Top-Level Testbench and Stimulus File

Below is the resulting waveform from a top-level test with opcode(s) = 0111 = SRL.

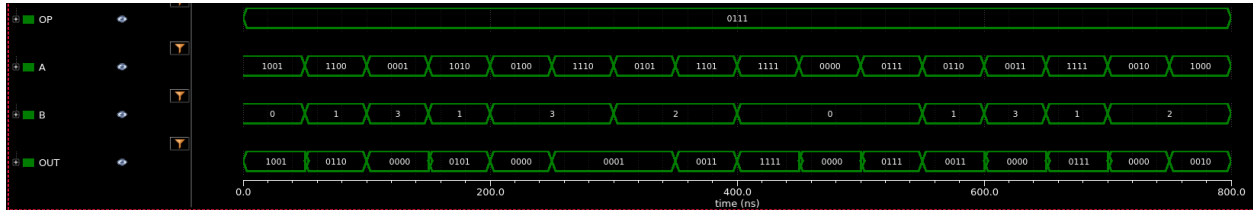


Figure 36: Top-Level Simulation Result for SRL

Below is a table listing the energy and delay simulations on the top schematic level. The rise and fall times were measured for the output's transition time, the propagation delay was measured between the 50% points of an input and corresponding output transition, and the clk to Q delay was measured from the time the clk transitioned and settled to the time the output transitioned and settled.

Subcell Operation	Energy (fJ/cycle)	T _{rise} (ps)	T _{fall} (ps)	T _{PD} (ps)	T _{CLK→Q} (ps)
AND	34.52	153.45	195.82	559.73	153.68
OR	34.77	163.87	198.93	580.37	204.28
XOR	34.8	159.24	203	532.06	223.78
ADD	34.7	211.17	192.72	528.8	151.08
SUB	48.16	167.07	154.85	456.71	91.342
MUL	34.67	166.82	206.73	254.89	72.7
SLL	34.5	231.57	184.84	449.62	136.12

SRL	34.56	178.9	182.86	388.15	88.43
CMP	34.41	135	151.37	455.9	130.67

Table 2: Simulation Performance Table – Schematics

After simulating using the extracted subcells, there was an increase in energy consumption and delays, due to the parasitics introduced. For larger layouts such as the bitwise top-level block, there were lots of long parallel metals introducing large capacitances and possible crosstalk. This is especially noticeable for the adder layout, which had the longest length and largest increase in energy and delays.

Subcell Operation	Energy (fJ/cycle)	T_{rise} (ps)	T_{fall} (ps)	T_{PD} (ps)	T_{CLK→Q} (ps)
AND	35.1	168.16	205.5	891.2	172.38
ADD	48.18	216.05	186.16	932.67	455.49
SRL	34.79	181.64	189.28	590.47	143

Table 3: Simulation Performance Table – Post-Extraction

DISCUSSION

Dynamic Logic: One consequence of our use of dynamic logic for the bitwise operations is that the output is only correct when the clock is high. One way we would like to refine our design if given more time would be to add latches to the outputs of the dynamic operations. This way, the signal labeled as output is always correct when read, rather than reflecting intermediate values during the low clock period.

Shifter Design: In the lecture slides, we were exposed to different types of shifters. When we first started schematic design, we designed all the options and compared them to each other based on performance. Logarithmic shifters have more benefit for much larger bit shifting, so we decided to move forward with the barrel rotator. Strangely, our shifters only pulled the output up to 0.6V instead of 1V despite many adjustments. To fix this, we created pass gates. Originally, a pure NMOS network became a passgate network to have a stronger pull-up. We tried to add more dynamic logic to it to make it faster, which worked, but it increased our power consumption a lot.

Carry Propagation: The outputs of the ripple carry adder use different types of logic. Carry-out used dynamic logic while sum used static logic; It required clock tuning (rise/fall time and period) and input alignment to ensure proper synchronization to have readable outputs.

Glitch Reduction: For the ripple carry outputs, we noticed that the signals were not clear nor stable. It was hard to see if it was working as it was intended. We found that the solution for this was to use x4 inverters to buffer the outputs. This mitigated the glitches, stabilizing both sum and

carry outputs. Interestingly enough, in the extracted simulations, the glitches disappeared despite showing up in the schematic simulations.

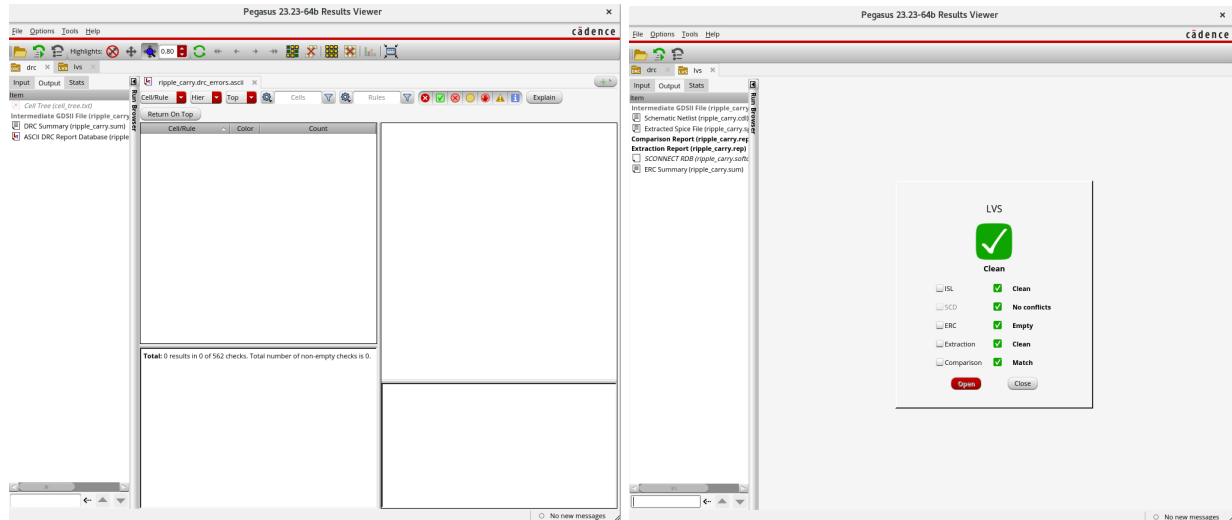
Functional Prioritization and Divider Consideration: When we were originally planning for this project, there were supposed to be 11 operations. We decided to deprioritize the divider in favor of completing core ALU integration. Dividing is not a staple operation, unlike shift and subtract, so it made more sense to focus on that and verify that functionality. Dividers are commonly implemented in separate floating-point units and may be explored in future extensions. If we had more time, that would have been an interesting thing to explore.

CITATIONS

- [1] Agarwal, A., & Sharma, R. (2013). Design and implementation of 4-bit ALU using reversible logic gates. In 2013 International Conference on Emerging Trends in Communication, Control, Signal Processing and Computing Applications (C2SPCA) (pp. 1–6). IEEE. <https://doi.org/10.1109/C2SPCA.2013.6718575>
- [2] Molnar, A. (2025). ECE 4740 Lecture Slides. Cornell University.
- [3] Razmkhah, S., & Bozbey, A. (2023). Efficient superconductor arithmetic logic unit for ultra-fast computing. arXiv. <https://doi.org/10.48550/arXiv.2312.09386>
- [4] Zhao, K. (2015). 4-bit ALU design: Final project report. University of Wisconsin–Madison. https://pages.cs.wisc.edu/~kzhao32/projects/ece467_4bitALU.pdf

APPENDIX: DRC + LVS SCREENSHOTS

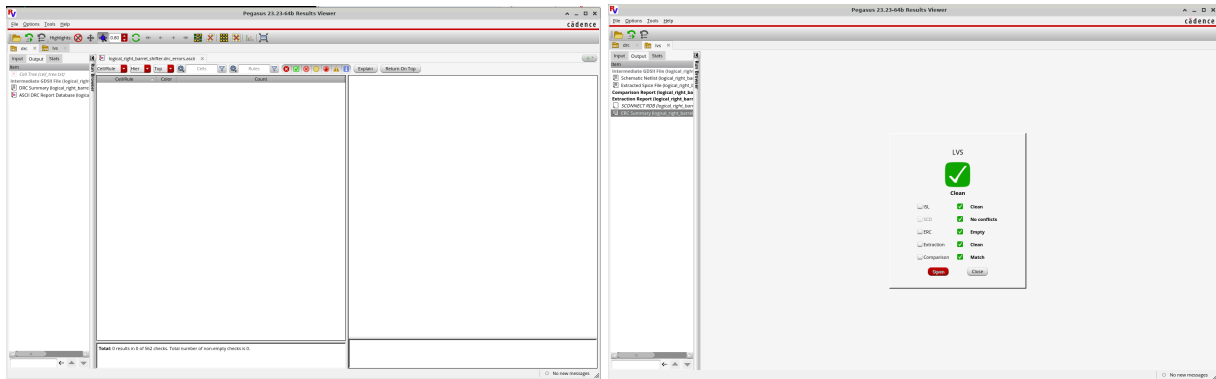
ADD:



SUB: (same circuit as add with inverters added in top cell)

MUL:

SRL:



SLL:

Bitwise Top Level:

